NeuroSky MindWave Mobile Apple SDK

Developer Guide · v1.0.0

Table of Contents

1. Overview
2. How It Works — Architecture
3. Requirements
4. Installation
5. Permissions
6. Quick Start
7. EEG Data Model
8. EEG Frequency Bands Explained
9. Signal Quality
10. Commands
11. Simulator — Develop Without Hardware
12. Error Handling & Reconnection
13. Advanced Patterns
14. Troubleshooting
15. Testing
16. API Reference

---

1. Overview

The **NeuroSky MindWave Mobile Apple SDK** is a modern Swift library that lets you read real-time EEG (electroencephalography) data from a NeuroSky MindWave Mobile headset on iOS 14+ and macOS 11+.

Key features

Feature	Description
BLE + BT Classic	iOS: BLE. macOS: BLE with automatic BT Classic fallback
Swift Concurrency	`AsyncStream<BrainWaveData>` — integrates naturally with SwiftUI `.task {}`
Built-in Simulator	Full data simulation without any hardware
SPM distribution	One-line Package.swift dependency
iOS 14+ / macOS 11+	Wide platform coverage

What you can measure

The MindWave Mobile headset contains a single dry electrode on the forehead (FP1 position) and a reference clip on the ear. From this single channel, the ThinkGear ASIC chip on board computes:

- **Raw EEG waveform** — 512 samples/sec, signed 16-bit values
- **8 frequency band powers** — Delta, Theta, Alpha (Low/High), Beta (Low/High), Gamma (Low/Mid)
- **eSense™ Attention** — NeuroSky's proprietary attention index (0~100)
- **eSense™ Meditation** — NeuroSky's proprietary relaxation index (0~100)

- **Signal quality** — 0 (perfect contact) to 200 (no signal)

---

2. How It Works — Architecture

```
┌──────────────────────────────────────────────────────────────────────────────┐
│         NeuroSky MindWave Mobile         │
│  ThinkGear ASIC chip                     │
│    → raw ADC samples (512Hz)             │
│    → computes FFT + eSense™ internally   │
│    → transmits via BLE or BT Classic     │
└──────────────────────────────┬───────────────────────────────────────────────┘
                               │ Bluetooth packets
                ┌──────────────▼──────────────┐
                │  Apple                      │
                │  Bluetooth APIs             │
                │  CoreBluetooth              │
                │  IOBluetooth                │
                └──────────────┬──────────────┘
                               │
                ┌──────────────▼──────────────────────────────────────────────┐
                │  NeuroSky MindWave Mobile Apple SDK                         │
                │                                                             │
                │  NeuroSkySdk (entry point)                                  │
                │    ├─── BLETransport                                        │
                │    │    CoreBluetooth GATT                                  │
                │    │    (CBCentralManager + callbacks)                      │
                │    ├─── BTClassicTransport (macOS only)                     │
                │    │    IOBluetooth RFCOMM SPP                              │
                │    │    (IOBluetoothRFCOMMChannel)                          │
                │    └─── SimulatorTransport                                  │
                │         (virtual data, no hardware)                         │
                │           ↓                                                 │
                │   ThinkGearParser                                           │
                │    decodes 0xEA / 0xEB / 0xEC packets                       │
                │    decodes raw EEG bytes                                    │
                │           ↓                                                 │
                │   BrainWaveData (emitted to AsyncStream)                    │
                └──────────────────────────────┬──────────────────────────────┘
                                               │ AsyncStream<BrainWaveData>
                                  ┌────────────▼────────────┐
                                  │  Your App               │
                                  │  for await { }          │
                                  └─────────────────────────┘
```

BLE vs BT Classic — internal differences

BLE (Bluetooth Low Energy) path — iOS + macOS: The MindWave Mobile exposes three BLE GATT characteristics. The SDK subscribes to notifications on the eSense (`039afff8`) and RawEEG (`039afff4`) characteristics, then writes the handshake command byte to the handshake characteristic (`039affa0`) to start data flow. No pairing is required.

BT Classic (RFCOMM SPP) path — macOS only: The MindWave Mobile emulates a serial port (SPP UUID `00001101-...`). The SDK opens an `IOBluetoothRFCOMMChannel` and reads a continuous byte stream. `ThinkGearParser` synchronizes on the `0xAA 0xAA` sync header. The device must be paired in macOS Bluetooth settings first.

Both paths produce identical `BrainWaveData` output through the same `dataStream`.

Auto-fallback strategy

`NeuroSkySdk.connect()` always tries BLE first. On macOS, if BLE does not reach `connected` within 5 seconds, it automatically disconnects BLE and retries with BT Classic. Your `dataStream` collection code is unchanged — the transport switch is transparent. On iOS, only BLE is used.

---

3. Requirements

Component	Minimum
iOS	14.0
macOS	11.0 (Big Sur)
Swift	5.7+
Xcode	14+
Bluetooth	BLE adapter (BLE mode) or Classic BT (BT Classic mode, macOS only)
Pairing	Not required for BLE; required for BT Classic

Supported headset

This SDK is designed and tested for the **NeuroSky MindWave Mobile 2**. Both BLE and BT Classic modes are supported on macOS. iOS supports BLE only.

---

4. Installation

Swift Package Manager — Xcode

1. Open your project in Xcode
2. File → Add Package Dependencies
3. Enter: `https://github.com/nsk-bci/mindwave-sdk-apple`
4. Select version rule: **Up to Next Major** from `1.0.0`
5. Click **Add Package**

Swift Package Manager — Package.swift

```
// Package.swift
dependencies: [
    .package(url: "https://github.com/nsk-bci/mindwave-sdk-apple", from: "1.0.0")
],
targets: [
    .target(
        name: "MyApp",
```

```
        dependencies: ["NeuroSkySDK"]
    )
]
```

Then run:

```
swift package resolve
swift build
```

---

5. Permissions

iOS — Info.plist

Add the Bluetooth usage description to your app's `Info.plist`:

```
<key>NSBluetoothAlwaysUsageDescription</key>
<string>Bluetooth is required to connect to the MindWave headset.</string>
```

iOS 13 and earlier also require `NSBluetoothPeripheralUsageDescription`, but this SDK targets iOS 14+ so only `NSBluetoothAlwaysUsageDescription` is needed.

CoreBluetooth will display a system permission dialog the first time your app initializes `CBCentralManager`. No additional runtime permission code is needed.

macOS — Info.plist + Entitlements

Info.plist:

```
<key>NSBluetoothAlwaysUsageDescription</key>
<string>Bluetooth is required to connect to the MindWave headset.</string>
```

App.entitlements (required for BT Classic via IOBluetooth):

```
<key>com.apple.security.device.bluetooth</key>
<true/>
```

> *Sandboxed macOS apps (required for Mac App Store distribution) must include `com.apple.security.device.bluetooth`. Without it, IOBluetooth calls will silently fail.*

---

6. Quick Start

SwiftUI example

```
import SwiftUI
import NeuroSkySDK

struct ContentView: View {
    let sdk = NeuroSkySdk()
```

```
    @State private var attention: Int = 0
    @State private var meditation: Int = 0
    @State private var signal: String = "–"

    var body: some View {
        VStack(spacing: 20) {
            Text("Attention: ₩(attention)")
                .font(.largeTitle)
            Text("Meditation: ₩(meditation)")
                .font(.largeTitle)
            Text("Signal: ₩(signal)")
                .foregroundColor(signal == "good" ? .green : .red)
        }
        .task {
            do {
                try await sdk.connect("MindWave Mobile")
                try await sdk.setNotch60Hz()

                for await data in sdk.dataStream {
                    attention  = data.attention
                    meditation = data.meditation
                    signal     = "₩(data.signalQuality)"
                }
            } catch {
                print("Connection error: ₩(error)")
            }
        }
    }
}
```

UIKit example

```
import UIKit
import NeuroSkySDK

class ViewController: UIViewController {

    let sdk = NeuroSkySdk()
    var streamTask: Task<Void, Never>?

    override func viewDidLoad() {
        super.viewDidLoad()
        startStreaming()
    }

    func startStreaming() {
        streamTask = Task { @MainActor in
            do {
                try await sdk.connect("MindWave Mobile")
                try await sdk.setNotch60Hz()
```

```
                for await data in sdk.dataStream {
                    guard data.signalQuality != .noSignal else {
                        statusLabel.text = "No signal — adjust headset"
                        continue
                    }
                    attentionLabel.text  = "Attention: ₩(data.attention)"
                    meditationLabel.text = "Meditation: ₩(data.meditation)"
                }
            } catch {
                statusLabel.text = "Error: ₩(error.localizedDescription)"
            }
        }
    }

    override func viewDidDisappear(_ animated: Bool) {
        super.viewDidDisappear(animated)
        streamTask?.cancel()
        Task { await sdk.disconnect() }
    }
}
```

Connecting by device name vs identifier

`connect()` accepts either a device name substring or a BLE peripheral UUID string:

```
// By device name (BLE scan required)
try await sdk.connect("MindWave Mobile")

// By BLE peripheral UUID (faster — skip scan)
try await sdk.connect("12345678-1234-1234-1234-123456789ABC")

// macOS BT Classic: by device name or Bluetooth address
try await sdk.connect("MindWave Mobile")
try await sdk.connect("AA:BB:CC:DD:EE:FF")
```

---

7. EEG Data Model

`BrainWaveData` is a value type (`struct`) emitted for every packet received from the headset.

```
public struct BrainWaveData {
    public let poorSignal: Int     // 0=perfect, 200=no signal
    public let attention: Int      // 0~100
    public let meditation: Int     // 0~100
    public let delta: Int          // 0.5~2.75 Hz
    public let theta: Int          // 3.5~6.75 Hz
    public let lowAlpha: Int       // 7.5~9.25 Hz
    public let highAlpha: Int      // 10~11.75 Hz
    public let lowBeta: Int        // 13~16.75 Hz
    public let highBeta: Int       // 18~29.75 Hz
```

```
    public let lowGamma: Int       // 31~39.75 Hz
    public let midGamma: Int       // 41~49.75 Hz
    public let rawEeg: [Int]       // 10 samples/packet at 512 Hz

    public var signalQuality: SignalQuality  // computed property
}

public enum SignalQuality {
    case good       // poorSignal == 0
    case fair       // poorSignal 1~50
    case poor       // poorSignal 51~199
    case noSignal   // poorSignal == 200
}
```

Packet rate

Data type	Rate
eSense (attention, meditation)	~1 Hz
EEG frequency bands	~1 Hz
Raw EEG	~51 packets/sec (10 samples × 51 = 510 samples/sec ≈ 512 Hz)

---

8. EEG Frequency Bands Explained

Band	Range	Associated with
Delta	0.5~2.75 Hz	Deep sleep, unconscious processes
Theta	3.5~6.75 Hz	Drowsiness, light sleep, creativity
Low Alpha	7.5~9.25 Hz	Relaxed, eyes-closed rest
High Alpha	10~11.75 Hz	Alert relaxation
Low Beta	13~16.75 Hz	Normal alert thinking
High Beta	18~29.75 Hz	Active thinking, concentration
Low Gamma	31~39.75 Hz	Cognitive binding, perception
Mid Gamma	41~49.75 Hz	Higher cognitive functions

The values are relative power units — compare them to each other (e.g., ratio of alpha to beta) rather than treating them as absolute measurements.

---

9. Signal Quality

`poorSignal` ranges from 0 (perfect contact) to 200 (no signal). The SDK maps this to a `SignalQuality` enum for convenience.

poorSignal	SignalQuality	Meaning
0	.good	Excellent electrode contact
1~50	.fair	Acceptable signal
51~199	.poor	Poor contact — reposition headset
200	.noSignal	No electrode contact detected

Recommended handling

```
for await data in sdk.dataStream {
    switch data.signalQuality {
    case .noSignal:
        showAlert("No signal — place the headset on your forehead and clip to your ear")
    case .poor:
        showWarning("Poor signal — adjust the headset position")
    case .fair, .good:
        updateUI(data)
    }
}
```

When `poorSignal == 200`, the attention and meditation values are meaningless (they will be 0). Always check signal quality before using eSense values.

---

10. Commands

Commands are sent to the headset via the handshake BLE characteristic after connection.

```
// Notch filter — call immediately after connecting
// Removes 50Hz or 60Hz power-line interference from raw EEG
try await sdk.setNotch60Hz()  // Korea, USA, Japan
try await sdk.setNotch50Hz()  // China, Europe, Australia

// Raw EEG stream — disabled by default to save bandwidth
try await sdk.startRawEeg()
// ... collect data.rawEeg ...
try await sdk.stopRawEeg()

// Low-level command access
try await sdk.sendCommand(NeuroSkyCommand.startEsense)
try await sdk.sendCommand(NeuroSkyCommand.stopEsense)
```

Command bytes reference

Constant	Value	Description
NeuroSkyCommand.startRawEeg	0x15	Enable raw EEG stream

NeuroSkyCommand.stopRawEeg	0x16	Disable raw EEG stream
NeuroSkyCommand.startEsense	0x17	Enable eSense stream (sent automatically on connect)
NeuroSkyCommand.stopEsense	0x18	Disable eSense stream
NeuroSkyCommand.notch50Hz	0x1B	50 Hz notch filter
NeuroSkyCommand.notch60Hz	0x1C	60 Hz notch filter

---

11. Simulator — Develop Without Hardware

`SimulatorTransport` generates realistic synthetic `BrainWaveData` at 1-second intervals without any Bluetooth hardware.

```
// Initialize SDK in simulator mode
let sdk = NeuroSkySdk(simulator: .focused)

Task {
    try await sdk.connect("sim")  // address string is ignored

    for await data in sdk.dataStream {
        print("Attention: ₩(data.attention)")  // 70-95 in .focused mode
    }
}
```

Available modes

Mode	attention	meditation	poorSignal	Use case
.random	20~80	20~80	0~10	General data flow testing
.focused	70~95	40~60	0	Focused state UI testing
.relaxed	20~45	70~95	0	Relaxed state UI testing
.poorSignal	0	0	200	Signal loss and error handling

The simulator is available in both debug and release builds. Use `#if DEBUG` to restrict it to development if desired.

---

12. Error Handling & Reconnection

Connection errors

`connect()` throws on failure. Wrap it in a `do-catch`:

```
Task {
    do {
        try await sdk.connect("MindWave Mobile")
    } catch BLEError.bluetoothUnavailable {
```

```
        showAlert("Bluetooth is off — enable it in Settings")
    } catch BLEError.connectionFailed {
        showAlert("Connection failed — is the headset on and nearby?")
    } catch BTError.deviceNotFound {
        showAlert("Device not found — pair it in macOS Bluetooth settings first")
    } catch TransportError.bleTimeout {
        // On macOS this is handled internally — SDK retries with BT Classic
        // This error only surfaces if BT Classic also fails
        showAlert("Could not connect via BLE or BT Classic")
    } catch {
        showAlert("Unexpected error: ₩(error)")
    }
}
```

Automatic reconnection

The SDK does not reconnect automatically after a disconnect. Implement reconnection in your app:

```
func connectWithRetry(maxAttempts: Int = 3) {
    Task {
        for attempt in 1...maxAttempts {
            do {
                try await sdk.connect("MindWave Mobile")
                return  // success
            } catch {
                if attempt < maxAttempts {
                    try? await Task.sleep(nanoseconds: 2_000_000_000)  // wait 2 sec
                } else {
                    showAlert("Failed after ₩(maxAttempts) attempts")
                }
            }
        }
    }
}
```

Observing connection state

```
Task {
    for await state in sdk.stateStream {
        switch state {
        case .scanning:    statusLabel = "Scanning..."
        case .connecting:  statusLabel = "Connecting..."
        case .connected:   statusLabel = "Connected"
        case .disconnected:
            statusLabel = "Disconnected"
            connectWithRetry()
        case .error(let err):
            statusLabel = "Error: ₩(err)"
        }
```

```
    }
}
```

---

13. Advanced Patterns

Combine integration

```
import Combine
import NeuroSkySDK

class EEGViewModel: ObservableObject {
    let sdk = NeuroSkySdk()

    @Published var attention: Int = 0
    @Published var meditation: Int = 0

    private var streamTask: Task<Void, Never>?

    func start() {
        streamTask = Task { @MainActor in
            try? await sdk.connect("MindWave Mobile")
            for await data in sdk.dataStream {
                self.attention  = data.attention
                self.meditation = data.meditation
            }
        }
    }

    func stop() {
        streamTask?.cancel()
        Task { await sdk.disconnect() }
    }
}
```

Raw EEG recording

```
var samples: [Int] = []

try await sdk.startRawEeg()

for await data in sdk.dataStream {
    samples.append(contentsOf: data.rawEeg)
    if samples.count >= 512 * 10 {  // 10 seconds of data
        analyzeEEG(samples)
        samples.removeAll()
    }
}
```

Frequency band ratios

```
for await data in sdk.dataStream {
    guard data.signalQuality == .good else { continue }

    let totalPower = data.delta + data.theta + data.lowAlpha +
                     data.highAlpha + data.lowBeta + data.highBeta

    guard totalPower > 0 else { continue }

    let alphaRatio = Double(data.lowAlpha + data.highAlpha) / Double(totalPower)
    let betaRatio  = Double(data.lowBeta  + data.highBeta)  / Double(totalPower)

    print("Alpha/Beta ratio: ₩(alphaRatio / betaRatio)")
}
```

---

14. Troubleshooting

"No signal" immediately after connecting

- Ensure the headset is on your forehead, not just held in your hand
- The metal electrode must touch your skin — push hair aside
- Moisten the ear clip contact slightly
- Try `poorSignal` simulator mode to confirm your UI handles `.noSignal` correctly

BLE connection times out on macOS

- The device may already be connected to another host — power cycle the headset
- Try moving within 1 meter of the headset
- Disable and re-enable Bluetooth in macOS System Settings
- If the issue persists, the SDK will automatically fall back to BT Classic (macOS only)

BT Classic "device not found" on macOS

- Open **System Settings → Bluetooth** and pair the MindWave Mobile before connecting
- The device must appear as paired (not just discovered) in the macOS Bluetooth list
- Ensure `com.apple.security.device.bluetooth` is in your app's entitlements

`@MainActor` concurrency warnings

`NeuroSkySdk` is isolated to `@MainActor`. If you see concurrency warnings, ensure you call it from a `@MainActor` context:

```
// Correct
Task { @MainActor in
    try await sdk.connect("MindWave Mobile")
}

// Also correct in SwiftUI
.task {
```

```
    try await sdk.connect("MindWave Mobile")
}
```

iOS Simulator

CoreBluetooth does not work in the iOS Simulator. Use `NeuroSkySdk(simulator:)` for development without a physical device:

```
#if targetEnvironment(simulator)
let sdk = NeuroSkySdk(simulator: .focused)
#else
let sdk = NeuroSkySdk()
#endif
```

---

15. Testing

The SDK ships with two test suites:

Unit tests — `ThinkGearParserTests`

Tests for the packet parser covering:

- `0xEA` packet: attention, meditation, poorSignal extraction
- `0xEB` packet: EEG frequency band 1/2
- Raw EEG: 10-sample parsing with sign correction
- Handshake packet: 20-byte structure and checksum

Integration tests — `SimulatorIntegrationTests`

Virtual device tests covering the full lifecycle:

Test	Verifies
`test_connect_yieldsConnectingThenConnected`	State sequence on connect
`test_disconnect_yieldsDisconnected`	State on disconnect
`test_focused_attentionInExpectedRange`	`.focused` mode value ranges
`test_relaxed_meditationInExpectedRange`	`.relaxed` mode value ranges
`test_poorSignal_signalQualityIsNoSignal`	`.poorSignal` mode + `.noSignal` quality
`test_rawEeg_has10SamplesPerPacket`	10 samples per packet
`test_rawEeg_samplesInValidRange`	Sample range (-512~512)
`test_receivesMultiplePacketsOverTime`	≥2 packets in 4 seconds
`test_sendCommand_doesNotThrow`	All commands succeed

Run tests:

```swift test
```

---

16. API Reference

`NeuroSkySdk`

```swift
@MainActor
public final class NeuroSkySdk {

    /// Stream of EEG data packets from the headset
    public let dataStream: AsyncStream<BrainWaveData>

    /// Stream of connection state changes
    public let stateStream: AsyncStream<ConnectionState>

    /// Initialize for real device connection
    public init()

    /// Initialize in simulator mode (no hardware required)
    public init(simulator mode: SimulatorTransport.Mode)

    /// Connect to headset by name or address
    /// - iOS: BLE only
    /// - macOS: BLE first, auto-falls back to BT Classic after 5 sec
    public func connect(_ deviceAddress: String) async throws

    /// Disconnect from headset
    public func disconnect() async

    /// Send a raw command byte to the headset
    public func sendCommand(_ command: UInt8) async throws

    // Convenience methods
    public func startRawEeg() async throws
    public func stopRawEeg() async throws
    public func setNotch50Hz() async throws
    public func setNotch60Hz() async throws
}
```

`ConnectionState`

```swift
public enum ConnectionState: Equatable {
    case scanning
    case connecting
    case connected
    case disconnected
```

```
    case error(Error)
}
```

`BrainWaveData`

```
public struct BrainWaveData {
    public let poorSignal: Int
    public let attention: Int
    public let meditation: Int
    public let delta: Int
    public let theta: Int
    public let lowAlpha: Int
    public let highAlpha: Int
    public let lowBeta: Int
    public let highBeta: Int
    public let lowGamma: Int
    public let midGamma: Int
    public let rawEeg: [Int]
    public var signalQuality: SignalQuality
}
```

`SignalQuality`

```
public enum SignalQuality: Equatable {
    case good       // poorSignal == 0
    case fair       // poorSignal 1~50
    case poor       // poorSignal 51~199
    case noSignal   // poorSignal == 200
}
```

`SimulatorTransport.Mode`

```
public enum Mode {
    case random      // randomized values
    case focused     // high attention, mid meditation
    case relaxed     // low attention, high meditation
    case poorSignal  // poorSignal = 200, all zeros
}
```

`NeuroSkyCommand`

```
public enum NeuroSkyCommand {
    public static let startRawEeg: UInt8  // 0x15
    public static let stopRawEeg: UInt8   // 0x16
    public static let startEsense: UInt8  // 0x17
    public static let stopEsense: UInt8   // 0x18
    public static let notch50Hz: UInt8    // 0x1B
```

```
    public static let notch60Hz: UInt8    // 0x1C
}
```

Error types

```
public enum BLEError: Error {
    case bluetoothUnavailable   // Bluetooth is off or restricted
    case connectionFailed       // GATT connection failed
    case deviceNotFound         // No matching peripheral found during scan
}

public enum BTError: Error {
    case deviceNotFound         // Not found in paired device list (macOS)
    case connectionFailed       // RFCOMM channel failed to open
}

public enum TransportError: Error {
    case bleTimeout             // BLE did not connect within 5 sec (macOS: retries BT Classic)
}
```